

Cupcake projekt



Gruppe D:

Nicolai Fællesskov : cph-nj322@stud.ek.dk : NicolaiFaellesskov, hold A

Gabriel Marcussen : cph-gm83@stud.ek.dk : Gabriel080103, hold B

Christopher Christensen : cph-cc355@stud.ek.dk : cuc2808, hold B

Peter Søndergaard : cph-ps338@stud.ek.dk : peter-kunosu, hold A

Indholdsfortegnelse

Forside:	1
Indholdsfortegnelse	2
Diverse Links:.....	2
Indledning	3
Baggrund.....	3
Teknologivalg.....	5
Krav	5
Figma, Opbygning og Diagrammer	7
Figma Skabelon.....	7
Klassediagram.....	8
ER diagram - DB struktur.....	9
Aktivitetsdiagram.....	10
Status på implementation	12
Proces	13

Diverse Links:

Link til Figma:

<https://www.figma.com/design/SwfQogxJvzoMPkuSA1Ls0x/Cupcake?m=auto&t=Mr m8zbiZnFKsDKAv-1>

DEMO Video:

<https://somup.com/cOfbjsVcRd2>

Indledning

Denne rapport er udarbejdet i forbindelse med 2. semesters prøveeksamen på datamatikeruddannelsen i Lyngby, hvor opgaven har været at udvikle en hjemmeside baseret på user stories samt ikke-funktionelle krav. Kravene er stillet af kunden Olsker Cupcakes, en cupcake butik på Bornholm, og omhandler udviklingen af en velfungerende prototype, der så vidt muligt opfylder de specificerede funktionelle behov og kan tages til videreudvikling eller et design team og til sidst deployment.

Rapporten præsenterer de opstillede krav fra kunden samt de teknologiske valg, der ligger til grund for udviklingen og afprøvningen af systemet. Derudover indgår diagrammer og designskabelon skabt i Figma, som har dannet fundament for projektets struktur og implementering. Afslutningsvis redegøres der for arbejdsprocessen, herunder developer gruppens mødedage, de identificerede fejl og mangler samt overvejelser vedrørende fremtidige forbedringer.

Baggrund

Som udgangspunkt ud fra opgaven har Olsker Cupcakes udarbejdet en række krav samt en foreløbig mock-up af en potentiel forside. Denne mockup fungerer som en overordnet visuel skitse, mens de tilhørende krav, formuleret som user stories, danner grundlag for de funktionelle egenskaber, hjemmesiden forventes at have.

På baggrund af dette er det projektgruppens opgave at omsætte kundens behov og kommunikation til konkrete og tekniske løsninger. Dette indebærer blandt andet at nedbryde kravene til mere håndterbare elementer, såsom en ordre bestående af cupcakes, som efterfølgende kan implementeres i objektorienteret kode.

Udviklingen af hjemmesiden omfatter desuden flere tekniske lag, herunder frontend, backend og database, som kommunikerer indbyrdes. I denne sammenhæng indgår blandt andet anvendelse af CRUD-operationer, der ikke har helt samme kode-syntax, opsætning eller samme logik, skønt de indtræder i flere led af programmet og ønsker at opnå det samme. Det er derfor ikke alene oversættelsen af kundens krav, der har en betydning, men også evnen til at skabe overblik og designe en optimal systemarkitektur. Af denne årsag er

der skabt og benyttet diagrammer samt en videreudviklet mock-up, som præsenteres senere i rapporten.

Nedenfor præsenteres gruppens fortolkning af kundens user stories, som danner grundlag for kravspecifikationen.

Fortolkning:

1. Der skal kunne bestilles cupcakes med valgfri bund og top.
2. Kunden skal kunne oprette en konto for at betale og gemme ordrer.
3. Der skal kunne oprettes en administratorkonto der kan ændre på kunde profiler og ordrer for at slette en ordre eller tilføje penge til en konto.
4. Kunden skal kunne se deres enkelte ordrelinjer i deres indkøbskurv med priser på inden de betaler.
5. Der skal kunne logges ind med e-mail og efter der er logget ind skal kundens/ administratorens e-mail vises i toppen af hjemmesiden på hver side.
6. Som administrator skal de kunne se alle ordrer i systemet.
7. Som administrator skal de kunne se alle kunde profiler i systemet og dermed også kunne rette deres saldo.
8. Som en kunde skal de kunne fjerne enkelte ordrelinjer i deres indkøbskurv for at kunne justere deres ordrer til.
9. Som administrator skal de kunne slette ugyldige ordrer f.eks. dem som ikke har betalt.

Note:

Fortolkningen afviger kun minimalt fra de oprindelige user stories, men er medtaget, idet den har dannet grundlag for gruppens videre udvikling med skabeloner og diagrammer.

Teknologivalg

Herunder præsenteres de specifikationer, som brugeren bør forholde sig til inden brug af programmet.

Brugte specifikationer:

IntelliJ IDEA Ultimate 2025.2.4.

Java 21.

Javalin 6.

Thymeleaf template engine.

Krav

Firmaets håb og projektgruppens rolle:

Formålet med systemet for Olsker Cupcakes er at gøre det nemmere for kunder at bestille cupcakes. Systemet giver desuden mulighed for at sammensætte individuelle kombinationer af bunde og toppings efter kundens ønsker, uden behov for fysisk kommunikation i butikken.

Med udgangspunkt i den stillede opgave har projektgruppen valgt at have primært fokus på funktionaliteten af hjemmesiden, da kundens krav i høj grad er tolket som værende funktionelt orienterede. Da ingen af gruppens medlemmer er uddannede designere, er der ikke lagt væsentlig vægt på det visuelle design. CSS er derfor anvendt i et begrænset omfang med fokus på at sikre en rimelig struktur, hvor elementer som knapper og medier fremstår funktionelle og brugbare.

Det er samtidig taget i betragtning, at et eksternt designteam potentielt kan overtage den visuelle del af løsningen på et senere tidspunkt. Fokus har derfor været på at udvikle en simpel og funktionel hjemmeside med korrekt implementeret funktionalitet frem for et færdigt visuelt design.

Kommende er de specifikke krav fra opgaven.

User Stories:

- US-1: Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, så jeg senere kan køre forbi butikken i Olsker og hente min ordre.
- US-2 Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en ordre.
- US-3: Som administrator kan jeg indsætte beløb på en kundes konto direkte i Postgres, så en kunde kan betale for sine ordrer.
- US-4: Som kunde kan jeg se mine valgte ordrelinjer i en indkøbskurv, så jeg kan se den samlede pris.
- US-5: Som kunde eller administrator kan jeg logge på systemet med e-mail og kodeord. Når jeg er logget på, skal jeg kunne se min email på hver side (evt. i topmenuen, som vist på mock-up'en).
- US-6: Som administrator kan jeg se alle ordrer i systemet, så jeg kan se hvad der er blevet bestilt.
- US-7: Som administrator kan jeg se alle kunder i systemet og deres ordrer, så jeg kan følge op på ordrer og holde styr på mine kunder.
- US-8: Som kunde kan jeg fjerne en ordrelinje fra min indkøbskurv, så jeg kan justere min ordre.
- US-9: Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde ugyldige ordrer. F.eks. hvis kunden aldrig har betalt.

De ikke funktionelle krav:

1. En mock up af hjemmesiden er lavet med Figma
2. Ordre, kunder, og andet data er gemt i en database.
3. Databasen skal normaliseres på 3. normalform med mindre andet giver mening.
4. Koden er delt på GitHub.
5. Det færdige produkt er udviklet i Java 21, Javalin, Thymeleaf template engine, postgres Database, HTML og CSS.
6. Hjemmesiden skal fungere på laptop og telefon.

Leverancer:

1. ERD over databasen.
2. En færdig hjemmeside som løser Olsker Cupcakes krav.
3. Alle krav skal løses slavisk fra 1 til 9.
4. Efter de første 6 krav er løst, skal der løses så mange af de sidste krav som nås.
5. En rapport over hele projektet og andet teknisk dokumentation.
6. En kort video demo af den færdige hjemmeside.

Figma, Opbygning og Diagrammer

I det følgende gennemgås den tidlige arbejdsproces, herunder opbygning af hjemmesiden i Figma, udarbejdelse af diagrammer samt opsætning af databasen. Disse aktiviteter har været grundlæggende for den struktur, der efterfølgende er anvendt i kodningen af systemet.

I gruppen er visualisering og overblik fordelt og efterfølgende præsenteret. De nedenstående elementer kan derfor i nogen grad afvige fra den endelige implementering, men de har dannet grundlag for det fælles overblik og er derfor inkluderet i rapporten.

Figma Skabelon

Olsker Cupcakes [Forside](#) [Bestil](#) [Min profil](#) [Admin](#)

Velkommen til Olsker Cupcakes

Bestil dine cupcakes online og hent dem i butikken i Olsker på Bornholm.

[Gå til bestilling](#)

Sådan fungerer det

1. Opret en konto eller log ind
 2. Vælg bund og topping
 3. Læg i kurven og betal
 4. Hent din ordre i Olsker
-

Om Olsker Cupcakes

Vi er et lille økologisk bageri fra Bornholm. Alle cupcakes er håndlavede med lokale råvarer.

Adresse: Olsker, 3770 Allinge, Bornholm

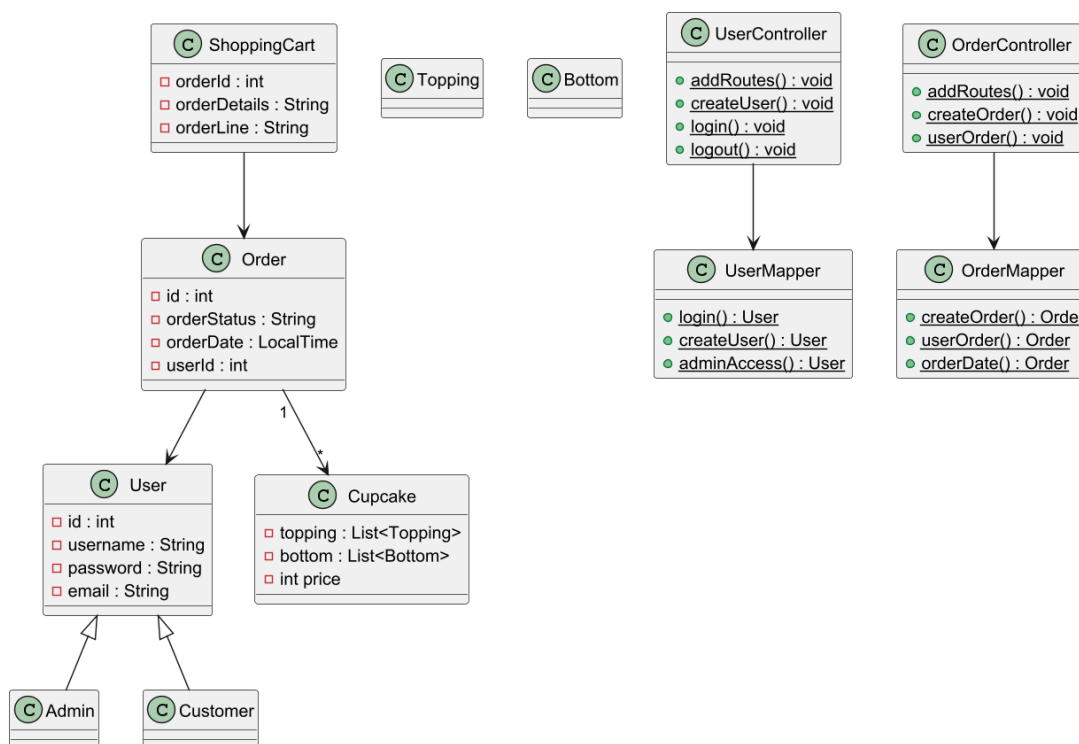
Email: olsker@cupcakes.dk

Link til Figma: Findes under indholdsfortegnelsen ved "Diverse Links".

Gruppens Figma skabelon/mockup har fungeret som det primære udgangspunkt for gruppens arbejde med koden. Der er blevet aftalt en skabelon med fokus på et simpelt til næsten intet design, så gruppen direkte har kunne arbejde med en ren hjemmeside uden visuel styling. Dette har gjort det muligt hurtigt at producere de forskellige HTML-sider og derefter fokusere på funktionaliteten uden at bruge tid på CSS i den tidlige fase.

Tilgangen har gjort det lettere at gå direkte til implementering af funktioner frem for at blive forsinket af designmæssige overvejelser. Samtidig har det givet et bedre overblik over strukturen i den kommende kode.

Klassediagram



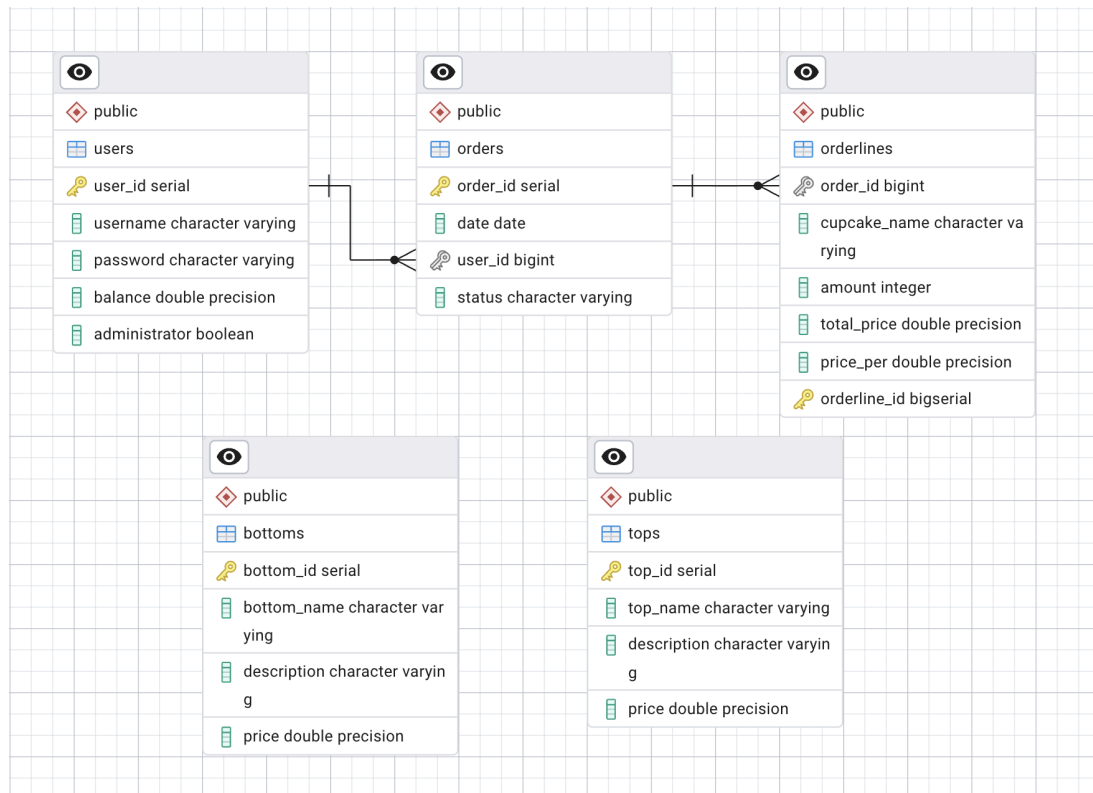
Klassediagrammet vises her som i udgangspunkt den oprindelige og eneste model, som er vist på ovenstående billede, og havde til formål at vise sammenhængen mellem systemets klasser. Diagrammet gav et godt overblik over de vigtigste entiteter, men blev ikke brugt i sin fulde form i den endelige løsning.

I stedet har det primært fungeret som et udgangspunkt for udviklingen, hvor klasserne løbende er blevet justeret. Klassen ShoppingCart er eksempelvis ikke implementeret som en separat klasse, men er i stedet håndteret gennem metoder i andre dele af systemet.

Der er desuden tilføjet en OrderLine-entitet samt en controller til administration af brugere med rollen Admin. Den oprindelige idé med sub- og superklasser, hvor Admin og Customer nedarver fra en fælles User-klasse, er blevet forenklet. I den endelige løsning anvendes i stedet én samlet User-klasse, hvor roller håndteres gennem attributter og logik. Dette skyldes primært en kombination af gruppens tidligere erfaring med klasser samt den nyere forståelse af udvikling på tværs af frontend og backend, herunder brug af controllere og

mappere. Derudover har arbejdet med koden i højere grad taget udgangspunkt i Figma og den bagvedliggende logik i funktionerne.

ER diagram - DB struktur



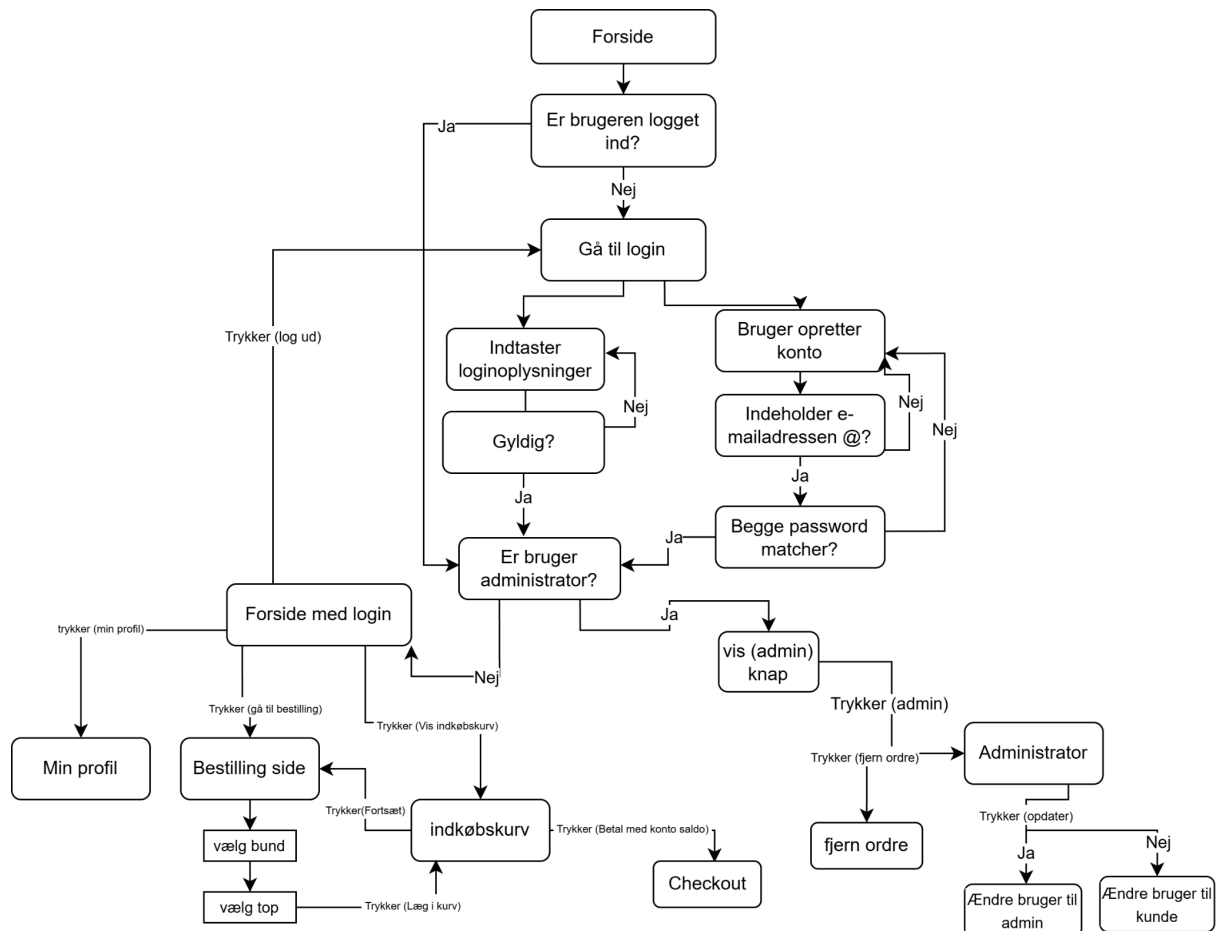
Her præsenteres gruppens ER-diagram samt et overblik over databasearkitekturen. Alle tabeller er designet med henblik på at opfylde 2. normalform. Selvom et af kravene var at opnå så høj grad af 3. normalform som muligt, opfylder tabellen **orderlines** ikke dette krav fuldt ud.

Dette skyldes den nære relation mellem **orders** og **orderlines**, som er nødvendig for at understøtte kundens krav om, at en ordre skal kunne bestå af flere ordrelinjer. Der er derfor foretaget et bevidst designvalg, hvor der er prioriteret funktionalitet og sammenhæng i datamodellen frem for fuld normalisering.

Tilsvarende kan tabellerne **tops** og **bottoms** i nogen grad betragtes som redundante i forhold til den endelige databaseimplementering. Disse anvendes dog indirekte, da backend håndterer sammensætningen af objekter og konverterer dem til en samlet cupcake-enhed, som efterfølgende indgår i **orderlines**. Dette er et bevidst arkitektonisk valg foretaget af gruppen for at sikre en mere fleksibel backend-logik.

Aktivitetsdiagram

Nedeunder er lagt et aktivitetsdiagram, som illustrerer forløbet i cupcake-systemet trin for trin, dette blev konstrueret senere i processen, men er inkluderet da det meget fint giver overblik over hjemmesiden. Under diagrammet er også lagt en forklaring. Det kan være et behov at læse med i forklaringen hvis man ikke har fuld forståelse diagrammet og ellers kan læseren selv vælge at analysere det.



Der er blevet brugt diagrams.net som værktøj til aktivitetsdiagrammet.

Processen starter på forsiden, hvor brugeren enten kan logge ind eller oprette en ny konto. Ved oprettelse eller login valideres brugerens input. E-mailadressen skal indeholde et "@", og hvis dette ikke er tilfældet, vises en fejlbesked. Ved oprettelse kontrolleres desuden, at de to indtastede adgangskoder er ens. Hvis valideringen ikke godkendes, får brugeren mulighed for at forsøge igen. Ved succesfuld oprettelse eller login oprettes en aktiv brugersession.

Efter login omdirigeres brugeren til forsiden, hvorfra der kan navigeres til systemets øvrige funktioner. Hvis brugeren er tildelt administratorrettigheder, gives der adgang til administrative funktioner.

Fra forsiden kan brugeren tilgå følgende områder: profilside, bestillingsside, indkøbskurv samt login/logud-funktion.

På profilsiden kan brugeren se og administrere sine egne oplysninger.

På bestillingssiden kan brugeren sammensætte en cupcake ved at vælge bund og topping, hvorefter produktet tilføjes til indkøbskurven. I indkøbskurven kan brugeren enten fortsætte bestillingen eller gå videre til checkout.

Ved checkout gennemføres bestillingen, hvorefter ordren afsluttes.

Administratorfunktionen giver mulighed for at administrere ordrer samt ændre brugerroller mellem kunde og administrator.

Særlige forhold

I dette afsnit gennemgås udvalgte kodebegreber, som er anvendt i projektet.

SessionAttributes

Et vigtigt bindeled mellem Java-koden og HTML er session attributes. Disse kan indeholde forskellige typer data og gemmes under et specifikt navn i en session.

Dette anvendes eksempelvis, når Java-koden har hentet data fra databasen og eventuelt oprettet et nyt objekt. Da dette objekt ikke direkte kan tilgås fra HTML/Thymeleaf, gemmes det i stedet som en session attribute. Thymeleaf kan herefter tilgå objektet via det angivne navn og anvende dets attributter og metoder.

Exceptions

Exceptions anvendes til at sikre, at systemet kan reagere på fejl bestemte fejl når det kører.

I programmet håndteres fejl ved at fange exceptions og vise en given fejlbesked, som enten kan være brugervenligt på hjemmesiden eller bag i consolen til programmøren. Et eksempel er ved tilføjelse af en ordre til indkøbskurven, hvor brugeren får en besked såsom: *“Der skete en fejl ved tilføjelse til kurv”*, hvis operationen ikke gennemføres korrekt.

Exceptions anvendes generelt i situationer, hvor der kan opstå uforudsete fejl under udførsel af programmet, for eksempel, ved databasekald eller ugyldigt input. På denne måde kan systemet reagere med en given reaktion og give feedback, så at fejlen kan identificeres og rettes.

Status på implementation

Kundekrav og hjemmeside flow:

Alle user stories og kundekrav er implementeret i systemet og fungerer som forventet. Hjemmesiden kører stabilt uden store problemer, og der er et sammenhængende flow, hvor brugeren kan navigere mellem de forskellige menupunkter.

Ses der på Figma-skabelonen er den endelige løsning i høj grad implementeret i som skabelonen, med få mindre afvigelser.

Intet styling og minimal CSS:

Der er lagt minimal fokus på styling i projektet, og CSS er holdt på et simpelt niveau. Dette gør det muligt for et eventuelt designteam at overtage og videreudvikle det visuelle udtryk i en senere fase.

Inputvalidering:

Validering af brugerinput er implementeret for at sikre, at brugeren indtaster gyldige data ved oprettelse og login.

Ved login kræves det, at brugeren anvender en gyldig e-mailadresse, en adgangskode på minimum 5 tegn samt korrekt bekræftelse af adgangskoden ved oprettelse.

Validering af e-mailadressen sker ved brug af HTML-inputtypen `type="email"`, som automatisk sikrer, at der indgår et @-tegn i inputtet.

For adgangskodefeltet anvendes HTML-validering med attributterne `minlength="5"` og `required`, som sikrer, at feltet ikke er tomt, og at adgangskoden har en minimumslængde på 5 tegn.

Derudover anvendes JavaScript til at kontrollere, at de to indtastede adgangskoder er ens ved oprettelse. Hvis de ikke matcher, vises en fejlbesked til brugeren.

DRY CODE:

Diverse HTML og Java kode mangler optimering, der mangler den fulde oprydning af DRY code.

Proces

I dette afsluttende afsnit redegøres der for gruppens arbejdsproces samt de overvejelser, der er gjort i forbindelse med projektet.

Arbejdsform og Projektforløb:

Arbejdsform:

Projektet er blevet tilrettelagt som et struktureret arbejdsforløb, hvor gruppen har arbejdet med faste rammer, herunder mødepligt, faste mødetidspunkter samt interne deadlines. Dette er gjort for at sikre høj arbejdsaktivitet og styrke kommunikationen i gruppen.

Den første dag adskilte sig fra det øvrige forløb, da den primært blev anvendt til at opstille rammerne for den kommende hjemmeside samt planlægning af arbejdsprocessen. Den videre arbejdsproces uddybes i de følgende afsnit.

Dagens proces:

Mødetid: 10:00 (senest 10:10).

Morgenmøde: 10:00 til 10:30.

Kodning/ Rapportskrivning: 10:30 til 14:00 (med pause klokken 12 eller 13).

Afslutningsmøde: 14:00 til 15:00.

- *Morgenmødet* blev anvendt til at gennemgå dagens mål samt diskutere de arbejdsopgaver, der skulle løses. Herefter fordelte gruppen sig i mindre arbejdsgrupper, hvor medlemmer med relaterede opgaver arbejdede sammen om lignende funktionalitet i koden.

- *Kodning/ rapportskrivning* blev anvendt til at arbejde med de opgaver, der var fastlagt på morgenmødet. Hvis en opgave ikke blev færdiggjort samme dag, blev den prioriteret som første opgave den følgende dag for at sikre et stabilt arbejdsflow og overholdelse af deadlines. Det fungerede særligt godt i forbindelse med arbejdet med user stories, hvor det blev anbefalet at arbejde systematisk og konsekvent med kravene.
- *Afslutningsmødet* blev anvendt til at opsummere dagens arbejde samt vurdere arbejdsindsatsen og fremdriften i forhold til dagens mål. Her blev det også drøftet, om de opstillede opgaver var blevet løst tilfredsstillende. Derudover blev der eventuelt lavet noter og planlægning til den efterfølgende arbejdsdag.

Dagens proces i praksis:

Da gruppen har haft fokus på at tilgå projektet som et professionelt arbejdsforløb, er der løbende blevet ført daglige noter over arbejdet. Disse er struktureret som en beskrivelse af dagens proces.

Nedenfor ses et eksempel fra en af arbejdsdagene:

“Plan torsdag d. 9. april:

Vi skal begynde at kode.

- *Vi skal have login på plads. Sådan at vi kan lave en sammenhæng mellem Currentsession og den bruger som bliver logget ind på.*

Hvad blev opnået i dag?:

- *Gabriel: Ordermapper og orderController er blevet lavet.*
- *Christophe & Nicolai: UserMapper, SQL og UserController er blevet lavet.*
- *Peter: index.html, login.html og order.html er blevet lavet + stylesheet er tilføjet for bedre læseligt html.*

Hvordan synes vi det gik i dag?:

- *Gabriel: Synes det er gået fornuftigt og ting bliver gjort til tiden.*
- *Christopher: Tilfreds med det som er blev opnået og der er blevet gjort klar til sammensætning af frontend og backend*
- *Nicolai: Synes det er gået godt.*
- *Peter: Vi er langt fremme med deadlines og der er stadig god struktur på hele projektet.*

Mødested: Primært mødested igen fredag klokken 10.”

Som det ses i eksemplet, har gruppen været tilfreds med arbejdsformen, som har bidraget til en rød tråd gennem hele projektet. Den klare struktur har gjort det muligt at fokusere på kodningen uden løbende at skulle tage stilling til, hvilke opgaver der skulle løses.

Rollefordeling:

Herunder er rollerne listet og gruppens fortolkning af dem under projektet.

Tech lead: Peter & Christopher

Scrum master: Gabriel

Product owner: Nicolai

Tech lead:

Tech lead er den rolle som har sidste ord omkring koden og bestemmer kodestandarden, men som regel også bør give sig ekstra tid til at holde styr på hele gruppens kodesammenhæng.

Scrum master:

Scrum master er hvad der ville kaldes for en projektleder, de sørger for at processen kører, det vil sige at der holdes møder omkring hvilke opgaver der skal laves på dagen.

Product owner:

Product owner er dem som snakker med kunden og oversætter kundens krav til tech lead og andre udviklere.

Evaluering og Overvejelser:

Hvad var godt og hvad var dårligt:

Gruppen er tilfreds med den planlagte arbejdsproces og det velfungerende resultat, som denne har ført til.

Et område, hvor gruppen vurderer, at der er potentiale for forbedring, er kommunikationen i forbindelse med kodningen. Gruppen oplevede, at der ofte var behov for, at et enkelt gruppemedlem arbejdede videre om aftenen med at rette fejl og sikre sammenhæng i koden.

Dette kunne i højere grad være undgået gennem bedre løbende koordinering samt større fokus på, at koden fungerede på tværs af Git-branches.

Hvad kan læres af processen og hvad kan ændres:

Et centralt læringspunkt i projektet har været betydningen af planlægning og fastsættelse af daglige mål i arbejdsprocessen. Dette har vist sig at være en effektiv metode til at skabe struktur og fremdrift.

Derudover har arbejdet med at skabe overblik over hele systemet – fra HTML til database – bidraget til en bedre forståelse af samspillet mellem de forskellige lag. Denne erfaring har styrket gruppens samlede tekniske forståelse.

Et forbedringsområde til fremtidige projekter er en bedre koordinering mellem kode-branches, så det sikres, at koden ikke blot kan merges, men også fungerer korrekt på tværs af metoder og funktioner. Dette kan være med til at undgå mindre fejl og uafsluttede opgaver ved dagens afslutning.

Så kan der med fordel lægges større fokus på optimering af koden, så unødvendig gentagelse undgås. Dette kan betyde mindre DRY kode, så metoder ikke ender med at være næsten identiske.

Som et yderligere forbedringspunkt kan det overvejes, at alle gruppemedlemmer i højere grad arbejder selvstændigt uden for den fælles arbejdstid. Dette giver mulighed for fordybelse i koden samt løbende forbedringer af dagens arbejde.

En sådan tilgang kan bidrage til øget læring for det enkelte gruppemedlem og samtidig skabe et bedre grundlag for faglig sparring og mere kvalificerede forslag i det videre gruppearbejde.